

ASSESSMENT: No SQL -MongoDB

Theme: E-Commerce product backend

Section A: conceptual understanding

1) Why node.js is preferred for real-time backend APIs?

Ans: node.js is preferred because:

It uses a single-threaded event-driven architecture.

Handles multiple client requests simultaneously without creating multiple threads.

Uses non-blocking i/o operations, making applications fast and scalable.

Suitable for:

Chat applications

Live notifications

Online gaming

e-commerce APIs

real-time dashboards

example: instead of waiting for one database query to finish, node.js continues handling other requests

2) Explain non-blocking i/o using a file read example

Ans: blocking I/O

```
Const fs = require("fs");
```

```
Const data = fs.readFileSync("sample.txt", "utf8");
```

```
Console.log(data);
```

```
Console.log("program ended");
```

Output: file content

Program ended

Node waits until the file is read

```
Const fs = require("fs");  
Fs.readFile("sample.txt", "utf8", (err, data) = {  
  Console.log(data);});  
Console.log("program ended");
```

Output:

Program ended

File content

Node continues executing while the file is being read

### 3) How promise prevent callback hell

Ans:

Callback hell

```
Getuser(id, function(user) {  
  Getorders(user.id, function(orders) {  
    Getpayment(orders..id, function(payment) {  
      Console.log(payment);  
    });});});
```

Nestend callbacks become difficult to read and maintain

Using promises

```
Getuser(id)  
.then(user => getorders(user.id))
```

```
.then(orders => getPayment(orders.id))  
.then(payment => console.log(payment))  
.catch(err => console.log(err));
```

Benefits:

Cleaner code

Better readability

Easier error handling

Supports async/await

#### 4) MongoDB document structure for product catalog

Ans: MongoDB stores data in json-like documents

Product document

```
{  
  "_id": "9987abc123",  
  "name": "nike shoes",  
  "price": 4999,  
  "description": "running shoes",  
  "category": "footwere",  
  "stock": 50  
}
```

Advantages:

Flexible schema

Easy nested objects

Faster development

## Horizontal scaling

### 5) Why aggregation pipelines are essential for reports

Ans: aggregation framework helps analyze and transform data

Example:

Calculate total stock by category.

```
Db.products.aggregate([
  { $group:{
    _id: "$category"
    totalStock: { $sum: "$stock" }
  }}]);
```

Benefits:

Reporting

Analytics

Dashboard generation

Data grouping

Summarization

### Section B: practical tasks

#### 1) Node.js server & routing

Install packages

Npm init -y

Npm install express mongoose

Server.js

```
Const express = require("express");
Const app = express();
App.use(express.json());
App.get("/products", (req, res) => {
  Res.json({
    Message: "product catalog API" });});
App.listen(3000, () => {
  Console.log("server running on port 3000");});
```

## 2) Request logging with fs module

```
Const fs = require("fs");
Const logger = (req, res, next) => {
  Const log = `${new date().toISOString()} |
  ${req.method} | ${req.url}\n`;
  Fs.appendFile("access.log", log, (err) => {
    If(err) {
      Console.log(err);
    });
  });
  Next();};
Module.exports = logger;
```

Use middleware

```
Const logger = require("./middleware/logger");
App.use(logger);
```

## 3) MongoDB CRUD operations

## Database connection

```
Const mongoose = require("mongoose");

Mongoose.connect("mongodb:
//127.0.0.1:27017/ecommerce")
.then(() => console.log("MongoDB connected"))
.catch(err => console.log(err));
```

## Product model

```
Const mongoose = require("mongoose");
Const productschema = new mongoose.schema({
Name:string,
Price: number,
Description: string,
Category: string,
Stock: number});
Module.exports =
mongoose.model("product",productSchema);
```

## Create product

```
App.post("/products", async (req, res) => {
Const product = await product.create(req.body);
Res.status(201).json(product);});
```

Read product

```
App.get("/products", async (req,res) => {  
  Const products = await product.find();  
  Res.json(products);});
```

Delete product

```
App.delete("/products/:id", async(req,res) =>{  
  Await product.findByIdAndDelete(req.params.id);  
  Res.json({  
    Message:"product deleted"}));});
```

#### 4) Advanced update operators

Update description using \$set

```
App.put("/products/:id", async(req, res) => {  
  Const product = await product.findByIdAndUpdate(  
    Req.params.id,  
    { $set:{  
      Description: req.body.description}},  
    {new:true});  
  Res.json(product);});
```

Increase stock using \$inc

```
App.patch("/products/:id/increase", async(req, res)=>{
```

```
Const product = await product.findByIdAndUpdate(
  Req.params.id, {
    $inc:{ stock:10}}, {new:true});
Res.json(product);
});
```

Decrease stock

```
App.patch("/products/:id/decrease", async (req, res) => {
  Const product = await product.findByIdAndUpdate(
    Req.params.id,
    {
      $inc:{
        Stock:-5}}, {new:true});
  Res.json(product);
});
```

Section C: mini project

Project inventory API

Project structure

project/

|

|— models/

| — Product.js



```
|  
| └─ middleware/  
|   └─ logger.js  
|     └─ errorHandler.js  
|  
| └─ routes/  
|   └─ productRoutes.js  
|  
| └─ access.log  
| └─ server.js  
| └─ package.json
```

project schema

```
const mongoose = require("mongoose");  
const productSchema = new mongoose.schema({  
  name:{  
    type:string,  
    required:true},  
  price: {  
    type:number,  
    required:true},  
  description:string,  
  category:string,  
  stock: {
```

```
type:number,  
default:0}});  
  
module.exports = mongoose.model("product",  
productSchema);
```

## CRUD routes

### Create

```
Route.post("/", async (req, res) =>{  
  Try{  
    Const product = await product.create(req.body);  
    Res.status(201).json(product);}  
  Catch(err) {  
    Res.status(500).json({error:err.message});});
```

### Read

```
Router.get("/", async (req, res) => {  
  Try {  
    Const products = await product.find();  
    Res.json(products);  
  }catch (err) {  
    Res.status(500).json({error:err.message});});
```

## Update

```
Router.put("/:id", async (req,res)=> {  
  Try{  
    Const product = await product.findbyidandupdate(  
      Req.params.id,  
      Req.body,  
      {new:true});  
    Res.json(product);  
  }catch (err){  
    Res.status(500).json({error:err.message});});
```

## Delete

```
Router.delete("/:id", async (req, res)=> {  
  Try {  
    Await product.findbyidanddelete(req.params.id);  
    Res.json({  
      Message: "deleted successfully"});  
  }catch(err){  
    Res.status(500).json({error: err.message });});
```

## Stock update logic

Restock product

```
Router.patch("/id/restock", async (req, res) => {  
  Const product = await product.findByIdAndUpdate(  
    Req.params.id,  
    { $inc: {  
      Stock: req.body.quantity}},  
    {new:true});  
  Res.json(product);});
```

Purchase product

```
Router.patch("/:id/purchase", async (req, res) => {  
  Const product = await product.findbyidandupdate(  
    Req.params.id,  
    {  
      $inc:{stock: -req.body.quantity}}, {new: true});  
  Res.json(product);});
```

Category aggregation

```
Router.get("/report/category", async (req, res) =>{  
  Const report = await = await product.aggregate([  
    {
```

```
$group: {  
  -id: "$category",  
  totalStock: {  
    $sum: "$stock"},  
  
  Totalvalue: {  
    $sum: {  
      $multiply: ["$proce", "$stock"]}}}});  
Res.json(report);  
});
```

### Sample Output

```
[  
  {  
    "_id": "Footwear",  
    "totalStock": 100,  
    "totalValue": 500000  
  },  
  {  
    "_id": "Electronics",  
    "totalStock": 50,  
    "totalValue": 1500000  
  }  
]
```

Global error handler

```
Const errorHandler = (err, req, res, next) => {  
  Res.status(500).json({  
    Success:false,  
    Message: err.message});};  
Module.exports = errorhandler;
```

Use middleware

```
App.use(errorHandler);
```

API Endpoints Summary

Method

Get: /products get all products

Get: /products/:id get single product

Post: /products create product

Put: /products/:id update product

Delete: /products/:id delete product

Patch: /products/:id/restock increase stock

Patch: /products/:id/purchase decrease stock

Get: /products/:id/purchase decrease stock

Get: /products/report/category category report

